UNIT I FIRST ORDER LOGIC

Key Concepts - Knowledge based Systems – Role of Logic - Syntax – Semantics – Interpretations – Denotation – Satisfaction and models – Pragmatics – Explicit and Implicit Beliefs - Logical Consequence – Expressing Knowledge – Knowledge Engineering - Basic and Complex Facts – Terminological Facts – Entailment – Abstract Individuals - Other Sorts of Facts.

1.1: Introduction

What is knowledge?

- ✔ ***Knowledge is an awareness of facts, a familiarity with individuals and situations, or a practical skill.***
- ✔ Knowledge of facts, also called propositional knowledge, is often characterized as true belief that is distinct from opinion or guesswork by virtue of justification.
- ✔ Knowledge can be produced in many ways. The main source of empirical knowledge is perception, which involves the usage of the senses to learn about the external world.
- ✔ Introspection allows people to learn about their internal mental states and processes.
- ✔ Other sources of knowledge include memory, rational intuition, inference, and testimony

What is a Knowledge Base?

- ✔ ***An AI knowledge base is a centralized digital repository designed to store, organize and provide access to information using AI.***
- ✔ Unlike conventional knowledge bases, an AI knowledge base is dynamic and interactive. It uses machine learning (ML) and natural language processing (NLP), enabling it to learn and evolve from user interactions and data input.
- ✔ This allows the AI knowledge base to improve its accuracy and relevance over time, adapting to the specific needs and patterns of its users.

1.1.1: Knowledge Based System (KBS)

A knowledge-based system (KBS) is a computer program that reasons and uses a centralized repository of data known as knowledge base to solve complex problems. A KBS includes a database of facts and a set of rules for how to use those facts to answer questions, make decisions, or solve problems.

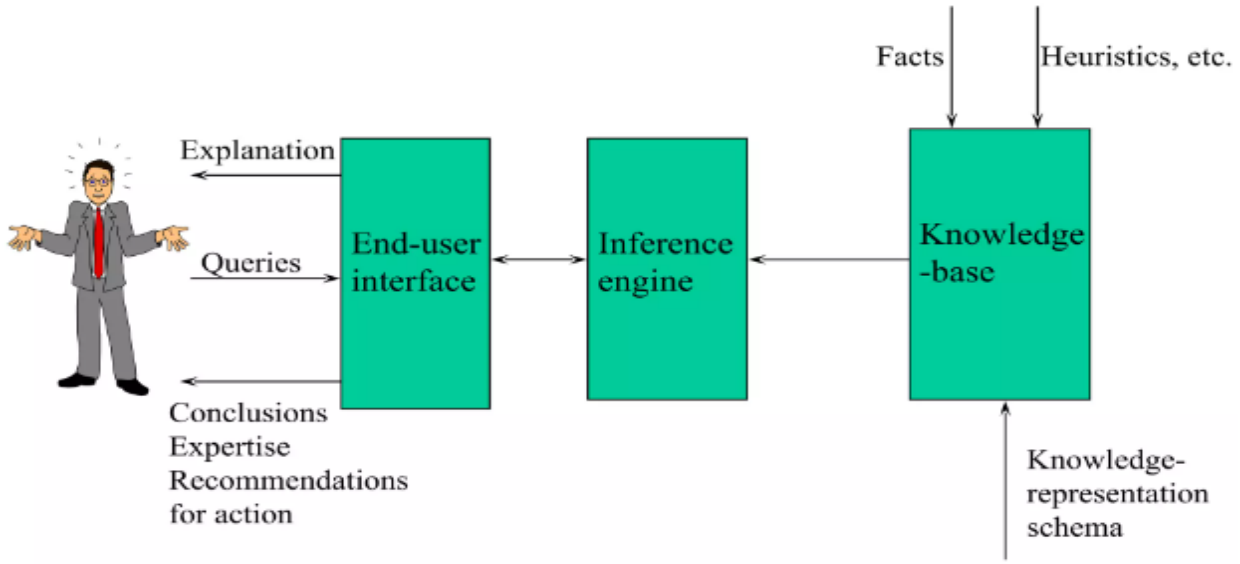
KBS are a form of AI designed to capture the knowledge of human experts to support decision-making.

- ***AIM of KBS:*** Replace human with a computer that contains all the relevant knowledge from a particular domain.

- ***KBS can assist in***
  - Decision making
  - Human learning
  - Creating a company-wide knowledge-sharing platform

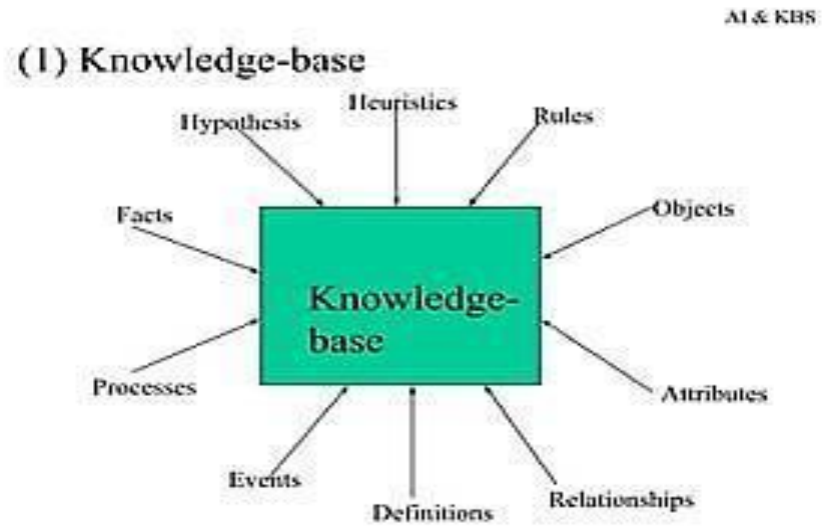
A) COMPONENTS OF KBS:

1. Knowledge Base
2. Inference Engine
3. User Interface



A basic KBS works using a knowledge base and an interface engine.

1) The **knowledge base** is a repository of data that contains a collection of information in a given field -- such as medical data.



Knowledge Representation:

- Knowledge is represented in a computer in the form of rules. Consists of an IF part and THEN part.
- IF part lists a set of conditions in some logical combination.
- If the IF part of the rule is satisfied; consequently, the THEN part can be concluded.
- Chaining of IF-THEN rules to form a line of reasoning.

2) The **inference engine** processes and locates data based on requests, similar to a search engine. A reasoning system is used to draw conclusions from data provided and make decisions based on if-then rules, logic programming or constraint handling rules.
3) Users interact with the system through a **user interface**.

B) DEVELOPING A KBS:

- ✔ Determining the characteristics of the problem
- ✔ Knowledge engineer and domain expert work together closely to describe the problem.
- ✔ The engineer then translates the knowledge into a computer-usable language, and designs an inference engine, a reasoning structure that uses the knowledge appropriately.

- ✔ He also determines how to integrate the use of uncertain knowledge in the reasoning process, and what kinds of explanation would be useful to the end user.

Advantages of KBS:

- ✔ Increase in available of expert knowledge
- ✔ Efficient and cost effective
- ✔ Consistency of answers
- ✔ Explanation of solution
- ✔ Deals with uncertainty

Disadvantages of KBS:

- Lack of common sense
- Inflexible, difficult to modify
- Restricted domain of expertise limited to KB
- Not always reliable

1.1.2: Types of knowledge-based systems

Some example types of knowledge-based systems include the following:

- **Blackboard systems.** These systems enable multiple sources to input new information into a system to help create solutions to potential problems. Blackboard systems rely heavily on updates from human experts.
- **Case-based systems.** These systems use case-based reasoning to create solutions to a problem. This system works by reviewing past data of similar situations.
- **Classification systems.** These systems analyze different data to understand its classification status.
- **Eligibility analysis systems.** These systems are used to determine a user's eligibility for a specific service. A system asks a user guided questions until it receives a disqualifying answer.
- **Expert systems.** These are a common type of KBS that simulate human expert decision-making in a particular field. Expert systems provide solutions for problems as well as the explanations behind them. For example, they could be used for calculations and predictions.

- **Intelligent tutoring systems.** These systems are designed to support human learning and education. Intelligent tutoring systems provide users with instructions and give feedback based on performance or questions.
- **Medical diagnosis systems.** These systems help diagnose patients by inputting data or having a patient answer a series of questions. Based on the responses, the KBS identifies a diagnosis and makes recommendations medical professionals can use to determine a patient's treatment.
- **Rule-based systems.** These systems rely on human-specified rules to analyze or change data to reach a desired outcome. For example, rule-based systems might use if-then rules.

1.2: Syntax

Syntaxes are the rules which decide how we can construct legal sentences in the logic.
It determines which symbol we can use in knowledge representation.
In FOL, there are two sorts of symbols:

Key Components of First-Order Logic

1. **Constants**:
   - **Definition**: Constants are symbols that represent specific objects in the domain.
   - **Examples**: If a, b, and c are constants, they might represent specific individuals like Alice, Bob, and Charlie.
2. **Variables**:
   - **Definition**: Variables are symbols that can represent any object in the domain.
   - **Examples**: Variables such as x, y, and z can represent any object in the domain.
3. **Predicates**:
   - **Definition**: Predicates represent properties of objects or relationships between objects.
   - **Examples**: P(x) could mean “x is a person”, while Q(x, y) could mean “x is friends with y”.
4. **Functions**:

- **Definition**: Functions map objects to other objects.
- **Examples**: f(x) could represent a function that maps an object x to another object, like "the father of x".

5. **Quantifiers**:
   - **Universal Quantifier (∀)**: Indicates that a statement applies to all objects in the domain. For example, ∀x P(x) means "P(x) is true for all x".
   - **Existential Quantifier (∃)**: Indicates that there exists at least one object in the domain for which the statement is true. For example, ∃x P(x) means "There exists an x such that P(x) is true".
6. **Logical Connectives**:
   - **Definition**: These include ∧ (and), ∨ (or), ¬ (not), → (implies), and ↔ (if and only if).
   - **Examples**: P(x) ∧ Q(x, y) means "P(x) and Q(x, y) are both true".
7. **Equality**:
   - **Definition**: States that two objects are the same.
   - **Examples**: x = y asserts that x and y refer to the same object.

Syntax of First-Order Logic:

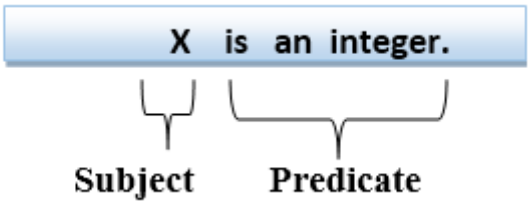
The syntax of **FOL** defines the rules for constructing well-formed formulas:

- **Atomic Formulas**: The simplest formulas, which can be predicates applied to terms.
  Example: Ravi and Ajay are brothers: => **Brothers(Ravi, Ajay).**
  Chinky is a cat: => **cat (Chinky)**.
- **Complex Formulas**: Formed by combining atomic formulas using **logical connectives and quantifiers**
  Example:
  Chinky is a cat and it likes fish: => **cat (Chinky) ∧ likes(fish)**

First-order logic statements can be divided into two parts:

- **Subject:** Subject is the main part of the statement.
- **Predicate:** A predicate can be defined as a relation, which binds two atoms together in a statement.

Consider the statement: "x is an integer.", it consists of two parts, the first part x is the subject of the statement and second part "is an integer," is known as a predicate.



Examples:

✔ "All purple mushrooms are poisonous"

$$\forall x\,(Mushroom(x) \wedge Purple(x) \rightarrow Poisonous(x))$$

Here's the breakdown:

- Mushroom(x): x is a mushroom.
- Purple(x): x is purple.
- Poisonous(x): x is poisonous.
- ∀x: For all x (for all objects x in the domain).

The statement asserts that for all x, if x is a mushroom and x is purple, then x is poisonous. This ensures that we explicitly refer to mushrooms in the context of the statement.

✔ "every gardener likes sun"

$$\forall x(Gardener(x) \rightarrow LikesSun(x))$$

Here's what each part of the statement means:

- Gardener(x): x is a gardener.
- LikesSun(x): x likes the sun.
- ∀x: For all x (for all individuals x in the domain).

Therefore, $\forall x\,(Gardener(x) \rightarrow LikesSun(x))$ asserts that for every individual x, if x is a gardener, then x likes the sun.

1.3: Semantics

Semantics refers to the meaning and interpretation of words, phrases, sentences, or symbols within a language or system. In the context of a knowledge base system, semantics involves the understanding and representation of knowledge in a way that machines can interpret meaningfully.

1.3.1: Importance of Semantics in Knowledge Base Systems

- ✔ **Understanding Context:** Semantics allows the system to understand the context of the information it stores, ensuring that queries can be answered accurately.
- ✔ **Interpreting Relationships:** It helps in understanding the relationships between different pieces of information, enabling the system to infer new knowledge from existing data.
- ✔ **Improving Accuracy:** By understanding the meaning of data, semantic systems can improve the accuracy and relevance of the information retrieved.
- ✔ **Facilitating Interoperability:** Semantics enables different systems to understand and work with each other's data, facilitating data sharing and integration across different platforms.

1.3.2: Key Components of Semantics in Knowledge Base Systems

- ✔ **Ontologies:** Ontologies are formal representations of a set of concepts within a domain and the relationships between those concepts. They are used to model domain knowledge in a structured form.

  Example: Ontology for a medical knowledge base might include concepts like "Disease," "Symptom," "Treatment," and the relationships between them.
- ✔ **Taxonomies:** A taxonomy is a hierarchical classification of entities. It helps in organizing knowledge by grouping similar entities under a common category.

  Example: In a library knowledge base, books might be categorized under genres like "Fiction," "Non-Fiction," "Science Fiction," etc.

- ✔ **RDF (Resource Description Framework):** RDF is a framework for representing information about resources in the web. It uses triples (subject, predicate, object) to make statements about resources.
  Example:
  Subject: "Alice"
  Predicate: "has favorite book"
  Object: "Moby Dick"
- ✔ **Rules and Inference Engines:** Rules are used to derive new information from existing data. Inference engines apply these rules to the knowledge base to generate new knowledge.

Example: If a person has a favorite book, then that book must exist in the system.

<u>Example of Semantic Representation</u>

Consider a simple knowledge base about people and their favorite books. Here's how semantics would be represented:

- Facts:
  - Alice is a person.
  - "Moby Dick" is a book.
  - Alice's favorite book is "Moby Dick."
- Ontology:
  - Concepts: Person, Book
  - Relationships: hasFavoriteBook
- RDF Triples:
  - (Alice, hasFavoriteBook, Moby_Dick)
  - (Moby_Dick, isA, Book)
  - (Alice, isA, Person)
- Rule:
  - If someone has a favorite book, that book must exist.
- Query:
  - What is Alice's favorite book?

1.4: Interpretations

An interpretation of a first-order language assigns a denotation to each non-logical symbol (predicate symbol, function symbol, or constant symbol) in that language.
It also determines a domain of discourse that specifies the range of the quantifiers. The result is that

- ✔ ***each term is assigned an object that it represents,***
- ✔ ***each predicate is assigned a property of objects, and***
- ✔ ***each sentence is assigned a truth value.***

In this way, an interpretation provides semantic meaning to the terms, predicates, and formulas of the language.

It involves specifying a domain of discourse D and a mapping I. Here's a detailed look at these concepts:

1. Domain of Discourse (D):
The domain of discourse, D, is the set of all objects that we are talking about in our logic system. Every variable in the logical formulas can take values from this domain.

2. Mapping (I):
Mapping I (also known as interpretation function) defines how the elements of the logical language are mapped to the elements in the domain of discourse. This includes the assignment of:

- **Constants:** Specific objects in the domain.
- **Functions:** Mappings from tuples of domain objects to domain objects.
- **Predicates:** Relations over the domain.

Formal Definition of Interpretation:

An interpretation I consist of:

1. **Domain D:** A non-empty set of objects.
2. **Interpretation Function I** It maps:
    - Each constant symbol c to a specific element $I(C) \in D$.
    - Each n-ary function symbol f to an n-ary function $I(f):D^n \rightarrow D$:

- Each n-ary predicate symbol P to an n-ary relation $I(P) \subseteq D^n$

Example:

Let's illustrate this with a simple example:

Domain D:

$D=\{1,2,3\}$

Constants:

- a is mapped to 1
- b is mapped to 2

Functions:

- f(x) is defined as x + 1 mod 3

Predicates:

- P(x) is true if x is even
- Q(x, y) is true if x < y

Detailed Mapping I:

- **Constants:**
  - I(a) = 1
  - I(b) = 2
- **Functions:**
  - $f^I(1) = 2$
  - $f^I(2) = 3$
  - $f^I(3) = 1$
- **Predicates:**
  - $I(P)=\{2\}$ (since 2 is the only even number in the domain)
  - $I(Q)=\{(1,2),(1,3),(2,3)\}$

Evaluating Formulas Under Interpretation I:

- **Atomic Formulas:**
  - P(a): Since I(a)=1,P(a) is false because $1 \notin I(P)$
  - P(b): Since I(b) =2, P(b) is true because $2 \in I(P)$.

- Q(a,b): Since (1,2) ∈ I(Q), Q(a,b) is true.

- **Complex Formulas:**
  - ∀x P(x): This statement is false because not all elements in D are even.
  - ∃x P(x): This statement is true because there exists an even number in D (i.e., 2).
  - ∀x ∃y Q(x,y): This statement is true because for every x in D, there is a y in D such that x<y.

1.5: Denotation

Denotation refers to the meaning or reference of symbols (terms, predicates, functions) within a particular interpretation. Understanding denotation is crucial for defining how knowledge is represented and manipulated in a knowledge-based system.

Denotation in First Order Logic:

1. Terms:

- **Constants**: Denote specific objects in the domain.
- **Variables:** Denote arbitrary objects in the domain (depending on the context or assignment).
- **Function Terms:** Denote objects obtained by applying functions to other terms.

2. Predicates:

- Denote relations among objects in the domain.

Formal Denotation:

Given a domain Dand an interpretation I:

- **Constants:**
  - A constant c denotes a specific object in D, denoted by I(C).
- **Variables:**
  - A variable x can denote any object in D. Its denotation depends on the variable assignment g (a function from variables to objects in D).
- **Function Symbols:**

- An n-ary function symbol f denotes a mapping $I(f):D^n \rightarrow D$. The denotation of a function term f(t1,t2,...,tn) is $I(f(I(t_1),I(t_2),\ldots,I(t_n))$.

- **Predicate Symbols:**
  - An n-ary predicate symbol P denotes a relation $I(P) \subseteq D^n$. An atomic formula P(t1,t2,...,tn) is true if and only if $(I(t_1),I(t_2),\ldots,I(t_n)) \in I(P)$.

Example:

Domain D:

D={1,2,3}

Constants:

- a denotes 1

Functions:

- f(x) denotes x + 1 mod 3

Predicates:

- P(x) denotes "x is even"
- Q(x, y) denotes "x < y"

Detailed Denotations:

- **Constant:**
  - a denotes 1, so I(a)=1.
- **Function Term:**
  - f(a) denotes I(f(I(a)) = I(f(1)=2
- **Predicate:**
  - P(a) denotes I(P(1)). Since $1 \notin \{2\}$, P(a) is false.
  - Q(a,b) denotes I(Q(1,2)). Since $(1,2) \in \{(1,2),(1,3),(2,3)\}$. Q(a,b) is true.

1.6: Satisfaction and models

In First Order Logic (FOL), satisfaction and models are fundamental concepts used to determine whether a logical formula is true under a given interpretation.

Satisfaction

Satisfaction in FOL is about whether a given interpretation $\mathcal{I}$ makes a formula ϕ true. If $\mathcal{I}$ satisfies ϕ, we write $\mathcal{I} \models \phi$. The rules for determining satisfaction depend on the structure of the formula.

Basic Components:

1. **Atomic Formulas:**
   - An atomic formula $P(t_1, t_2, ..., t_n)$ is satisfied by $\mathcal{I}$ if $(t_1^{\mathcal{I}}, t_2^{\mathcal{I}}, ..., t_n^{\mathcal{I}}) \in P^{\mathcal{I}}$.
2. **Complex Formulas:**
   - **Negation:** $\mathcal{I} \models \neg\phi$ if and only if $\mathcal{I} \not\models \phi$.
   - **Conjunction:** $\mathcal{I} \models \phi \wedge \psi$ if and only if $\mathcal{I} \models \phi$ and $\mathcal{I} \models \psi$.
   - **Disjunction:** $\mathcal{I} \models \phi \vee \psi$ if and only if $\mathcal{I} \models \phi$ or $\mathcal{I} \models \psi$.
   - **Implication:** $\mathcal{I} \models \phi \rightarrow \psi$ if and only if $\mathcal{I} \not\models \phi$ or $\mathcal{I} \models \psi$.
   - **Biconditional:** $\mathcal{I} \models \phi \leftrightarrow \psi$ if and only if $\mathcal{I} \models \phi$ and $\mathcal{I} \models \psi$ or $\mathcal{I} \not\models \phi$ and $\mathcal{I} \not\models \psi$.
3. **Quantified Formulas:**
   - **Universal Quantifier:** $\mathcal{I} \models \forall x\, \phi(x)$ if for every $d \in D$, $\mathcal{I}[x \mapsto d] \models \phi(x)$. Here, $\mathcal{I}[x \mapsto d]$ denotes the interpretation that maps x to d.
   - **Existential Quantifier:** $\mathcal{I} \models \exists x\, \phi(x)$ if there exists some $d \in D$ such that $\mathcal{I}[x \mapsto d] \models \phi(x)$.

Models

A **model** of a set of formulas Σ is an interpretation $\mathcal{I}$ that satisfies every formula in Σ. If $\mathcal{I}$ is a model of Σ, we write $\mathcal{I} \models \Sigma$.

- **Satisfiability:** A set of formulas Σ is satisfiable if there exists at least one interpretation $\mathcal{I}$ such that $\mathcal{I} \models \Sigma$.
- **Validity:** A formula ϕ is valid if $\mathcal{I} \models \phi$ for every interpretation $\mathcal{I}$.

Satisfaction and Models: A Simple Example

To illustrate satisfaction and models in First Order Logic (FOL), let's use a straightforward example with concrete statements.

Domain of Discourse

Let's consider the domain $D = \{1, 2\}$.

Interpretation

We define an interpretation $\mathcal{I}$ as follows:

- **Constants:** $a^{\mathcal{I}} = 1$
- **Predicate:** $P^{\mathcal{I}} = \{1\}$ (i.e., $P(x)$ is true if $x = 1$, and false otherwise)

Formulas

Consider the following formulas:

1. $P(a)$
2. $\forall x\, P(x)$

Checking Satisfaction

Formula 1: $P(a)$

- **Interpretation:**
  - $a^{\mathcal{I}} = 1$

- **Evaluation:**
  - $P(a)$ means $P(1)$
  - Since $1 \in P^{\mathcal{I}}$, $P(1)$ is true
- **Conclusion:**
  - $\mathcal{I} \models P(a)$

Formula 2: $\forall x\, P(x)$

- **Interpretation:**
  - $D = \{1, 2\}$
- **Evaluation:**
  - For $x = 1$, $P(1)$ is true because $1 \in P^{\mathcal{I}}$
  - For $x = 2$, $P(2)$ is false because $2 \notin P^{\mathcal{I}}$
- **Conclusion:**
  - $\mathcal{I} \not\models \forall x\, P(x)$

Model

A model of a set of formulas Σ is an interpretation $\mathcal{I}$ that satisfies every formula in Σ.

Given Set of Formulas

Let's consider the set of formulas $\Sigma = \{P(a), \forall x\, P(x)\}$.

- **Checking the Interpretation $\mathcal{I}$:**
  - From our previous evaluation, $\mathcal{I} \models P(a)$
  - However, $\mathcal{I} \not\models \forall x\, P(x)$

Since $\mathcal{I}$ does not satisfy all formulas in Σ, $\mathcal{I}$ is not a model for Σ.

Finding a Model

To find an interpretation that satisfies Σ, consider:

1. $P(a)$ requires $P(1)$ to be true.
2. $\forall x\, P(x)$ requires $P(x)$ to be true for all $x \in D$.

Adjusted Interpretation

Define a new interpretation $\mathcal{I}'$:

- **Predicate:** $P^{\mathcal{I}'} = \{1, 2\}$

Checking the New Interpretation

- **Formula 1:**
  - $a^{\mathcal{I}'} = 1$
  - $P(a)$ means $P(1)$
  - $1 \in P^{\mathcal{I}'}$, so $P(1)$ is true
  - $\mathcal{I}' \models P(a)$
- **Formula 2:**
  - For $x = 1$, $P(1)$ is true because $1 \in P^{\mathcal{I}'}$
  - For $x = 2$, $P(2)$ is true because $2 \in P^{\mathcal{I}'}$
  - $\mathcal{I}' \models \forall x\, P(x)$

Since $\mathcal{I}'$ satisfies both formulas, $\mathcal{I}'$ is a model for Σ.

Conclusion

Satisfaction determines if a specific interpretation makes a formula true, while a **model** is an interpretation that satisfies all formulas in a set. In our example:

- $\mathcal{I}$ satisfies $P(a)$ but not $\forall x\, P(x)$.
- $\mathcal{I}'$ satisfies both $P(a)$ and $\forall x\, P(x)$, making it a model for the set of formulas Σ.

1.7: Pragmatics

Pragmatics deals with the context-dependent aspects of meaning which are not captured by formal semantics alone. This includes:

- **Context:** Information about the situation in which a statement is made.
- **Speaker Intent:** The purpose or intent behind a statement.
- **Conversational Implicature:** Meaning that is implied but not explicitly stated.

1.7.1: Integrating Pragmatics into FOL

Integrating pragmatics into FOL for knowledge-based systems involves enhancing the system's ability to interpret and generate statements based on context, speaker intent, and other practical considerations.

Example

Imagine a knowledge-based system in a smart home environment. The system uses FOL to represent and reason about various states and actions within the home. By integrating pragmatics, the system can:

- ✓ **Contextual Understanding:** Recognize that "turn on the light" in the evening refers to the living room light, while the same command in the morning might refer to the bedroom light.

- ✓ **Intent Recognition:** Understand that when a user says "It's too cold," they likely intend for the heating to be increased.

- ✓ **Conversational Implicature:** Infer that a request like "Can you play some music?" typically implies playing the user's favorite playlist unless specified otherwise.

By incorporating pragmatics into FOL and knowledge-based systems, we can create more intelligent, context-aware, and user-friendly systems.

Practical Example:

Scenario

User: "Can you turn on the light?"

Context: The user is in the living room, and it is evening.

First-Order Logic Representation

1. **Basic Definitions:**
   - Light(x): Predicate indicating that x is a light.
   - TurnOn(x): Predicate indicating the action of turning on x.
   - InRoom(y, z): Predicate indicating that y (user) is in room z.
   - CurrentTime(t): Predicate indicating the current time t.
   - Evening(t): Predicate indicating that t is evening.
2. **Knowledge Base:**
   - Light(livingRoomLight): The light in the living room.
   - InRoom(user, livingRoom): The user is in the living room.
   - CurrentTime(t) ∧ Evening(t): It is currently evening.

Pragmatic Enhancements

1. **Contextual Information:**
   - The system understands the user's current location (livingRoom) and the time of day (Evening).
2. **Intent Recognition:**
   - The system infers that the user wants the light in the current room (living room) to be turned on.

Logical Steps to Process the Request

1. **User Request Interpretation:**
   - The request "Can you turn on the light?" is interpreted as TurnOn(x) where Light(x) and InRoom(user, z).
2. **Contextual Matching:**
   - Given InRoom(user, livingRoom) and Light(livingRoomLight), the system matches the light to the room.
3. **Action Execution:**
   - The system performs TurnOn(livingRoomLight)

Example in FOL with Pragmatics

Let's formalize this process using FOL:

1. Identify the Light in the Room:
   InRoom(user, livingRoom) ∧ Light(livingRoomLight) → TurnOn(livingRoomLight)

2. Contextual Information:
   CurrentTime(t) ∧ Evening(t)

3. User Request:
   UserRequest(TurnOn(Light))

4. Inference Rule:
   UserRequest(TurnOn(Light)) ∧ InRoom(user, Room) ∧ Light(RoomLight) ∧ CurrentTime(t) ∧ Evening(t) → TurnOn(RoomLight)

1.8: Implicit and Explicit Belief

In knowledge-based systems, beliefs are critical components that represent the system's understanding or assumptions about the world.
These beliefs can be categorized into explicit and implicit beliefs.

1. **Explicit Belief**
   Explicit beliefs are those that are directly represented within the knowledge base of a system. These are beliefs that the system has been explicitly told or that it has directly inferred from its data.
   Examples include:
   - ✓ Facts and Rules: Statements that are stored in a database or rule base, such as "All humans are mortal" or "Socrates is a human."
   - ✓ Direct Assertions: Data entries that explicitly state certain information, like "The temperature is 25°C."

   Characteristics:
   - ✓ Direct Representation: Explicit beliefs are clearly defined and stored within the system.
   - ✓ Easily Queryable: These beliefs can be easily accessed and retrieved by querying the knowledge base.
   - ✓ Clear Source: The origin of explicit beliefs is often clear, as they are input directly by users or derived through explicit inference rules.

2. Implicit Belief

Implicit beliefs are not directly represented but can be inferred from explicit beliefs using reasoning processes. These are beliefs that the system understands based on the information it has but are not explicitly stated in the knowledge base.

Examples include:

- ✓ Inferred Information: If the system knows "All humans are mortal" and "Socrates is a human," it can infer the implicit belief "Socrates is mortal."
- ✓ Contextual Understanding: Understanding implied meaning or context, such as recognizing that "It's hot today" implies a high temperature without it being explicitly stated.

Characteristics:

- ✓ Derived Through Inference: Implicit beliefs are obtained by applying logical inference rules to explicit beliefs.
- ✓ Context-Dependent: Often rely on the context and require additional reasoning to be extracted.
- ✓ Indirect Representation: These beliefs are not stored directly but are understood or generated as needed.

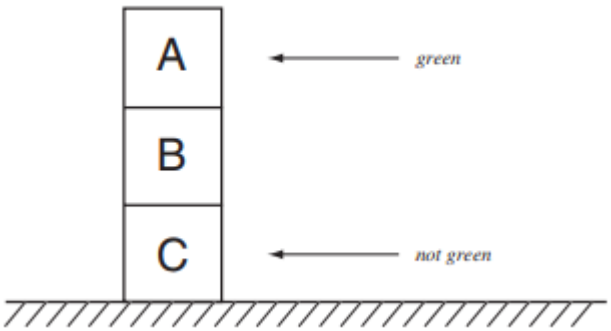
Practical Example

Consider a knowledge-based system in a medical diagnosis application:

- **Explicit Beliefs:**
  - Explicit Fact: "Patient A has a fever."
  - Explicit Rule: "If a patient has a fever, they may have an infection."
- **Implicit Beliefs:**
  - Implicit Inference: Given the explicit fact and rule, the system can infer, "Patient A may have an infection."
  - Implicit Contextual Understanding: If the system has contextual information that it is flu season, it might further infer, "Patient A may have the flu."

Illustrative Example

Consider the "blocks-world" example illustrated in the below Figure. Suppose we have three colored blocks stacked on a table, where the top one is green, the bottom one is not green, and the color of the middle block is not known. The question to consider is whether there is a green block directly on top of a nongreen one.



Information Given:

- The top block is green.
- The bottom block is not green.
- The color of the middle block is not known.

Block Arrangement:

- Top Block (A): Green
- Middle Block (B): Unknown color
- Bottom Block (C): Not green

Objective:

To find out if there is a green block directly on top of a non-green one.

Solution:

Formal Representation Using First-Order Logic (FOL):

To formalize this scenario in FOL, we define the following predicates:

- Green(x): x is green.
- OnTop(x, y): x is directly on top of y.
- NotGreen(x): x is not green.

The given information can be represented as:

1. Green(A)
2. OnTop(A, B)

3. OnTop(B, C)
4. NotGreen(C)

To determine if there exists a block x such that Green(x) and OnTop(x, y) and NotGreen(y), we use:

$$\exists x\ \exists y\ (Green(x) \land OnTop(x, y) \land NotGreen(y))$$

By the given facts, we can infer:

- ✓ Green(A) ∧ OnTop(A, B) (but color of B is unknown)
- ✓ Since NotGreen(C) ∧ OnTop(B, C), if Green(B), then OnTop(B, C) ∧ NotGreen(C).

Therefore, the explicit answer is:

The condition is satisfied if B is green, meaning there is indeed a green block directly on top of a non-green one.

1.9: Expressing Knowledge

Expressing knowledge in a knowledge-based system involves representing various types of facts and relationships using logical and formal structures.

1.9.1: Basic Facts

Definition: Basic facts are atomic assertions about entities and their properties or relationships. They are the simplest form of knowledge representation.

Formal Representation: In first-order logic (FOL), basic facts are typically represented using predicates, constants, and variables.

Example:

- **Unary Predicate**: Person(John) - Indicates that "John" is a person.
- **Binary Predicate**: Age(John, 30) - Indicates that "John is 30 years old".

Implications: Basic facts form the foundational building blocks upon which more complex knowledge structures are built. They are essential for defining the immediate properties of entities and their straightforward relationships.

1.9.2: Complex Facts

Definition: Complex facts involve more sophisticated relationships between entities and often use logical operators to combine basic facts.

Formal Representation: Complex facts are constructed using logical connectives such as AND (∧), OR (∨), NOT (¬), and IMPLIES (→).

Examples:

- **Conjunction (AND)**: Parent(John, Mary) ∧ Parent(Mary, Alice) - John is Mary's parent, and Mary is Alice's parent.
- **Disjunction (OR)**: Friend(Alice, Bob) ∨ Colleague(Alice, Bob) - Alice is either a friend or a colleague of Bob.
- **Negation (NOT)**: ¬Rich(Bob) - Bob is not rich.
- **Implication (IMPLIES)**: ∀x (Bird(x) → CanFly(x)) - If x is a bird, then x can fly.

Implications: Complex facts allow the representation of intricate dependencies and conditional relationships between entities. They are crucial for tasks requiring sophisticated reasoning, such as planning and diagnosis.

1.9.3: Terminological Facts

Definition: Terminological facts define the relationships between concepts within a domain. They are essential for building ontologies and taxonomies that structure domain knowledge.

Formal Representation: Terminological facts are often represented using subclass and instance relationships in description logics.

Examples:

- **Subclass Relationship**: ∀x (Dog(x) → Animal(x)) - All dogs are animals.

- **Instance Relationship**: Dog(Fido) - Fido is an instance of the class Dog.

Implications: Terminological facts enable the system to understand and reason about hierarchies and classifications, which are fundamental for semantic interoperability and knowledge integration across different systems and domains.

1.9.4: Entailment

Definition: Entailment is the logical consequence that can be inferred from a set of premises. If a conclusion logically follows from premises, it is said to be entailed by them.

Formal Representation: In formal logic, entailment is denoted as $\Gamma \models \phi$ meaning that the conclusion ϕ is a logical consequence of the premises Γ.

Example:

- **Premises**:
  - $\forall x$ (Bird(x) → CanFly(x)) - All birds can fly.
  - Bird(Tweety) - Tweety is a bird.
- **Entailed Conclusion**:
  - CanFly(Tweety) - Tweety can fly.

Implications: Entailment is central to reasoning processes in knowledge-based systems. It allows for the derivation of new knowledge from existing facts, supporting inference engines in decision-making and problem-solving.

1.9.5: Abstract Individuals

Definition: Abstract individuals are entities that do not have a physical manifestation but are essential for formal reasoning and mathematical computations. These include concepts like numbers, sets, and abstract objects.

Formal Representation: Abstract individuals are represented in a similar manner to other entities but are usually treated differently in terms of their logical properties and operations.

Examples:

- **Number**: Number(3) - 3 is a number.
- **Set**: Set({a, b, c}) - {a, b, c} is a set.

Implications: Abstract individuals allow systems to perform a wide range of reasoning tasks, including mathematical calculations, set operations, and logical manipulations, thereby extending the system's capabilities beyond tangible objects.

1.9.6: Other Sorts of Facts

(A) Temporal Facts

Definition: Temporal facts involve information related to time, capturing when events occur or the temporal ordering of events.

Formal Representation: Temporal logic, such as Linear Temporal Logic (LTL) or Computation Tree Logic (CTL), is used to represent temporal facts.

Examples:

- **Temporal Predicate**: HappensAt(Meeting, 9AM) - The meeting happens at 9 AM.
- **Temporal Ordering**: Before(9AM, 12PM) - 9 AM is before 12 PM.

Implications: Temporal facts are crucial for applications that require reasoning about time, such as scheduling, planning, and temporal databases.

(B) Spatial Facts

Definition: Spatial facts involve spatial relationships, such as location and adjacency, allowing reasoning about physical spaces.

Formal Representation: Spatial logic and geographic information systems

(GIS) often represent spatial facts.

Examples:

- **Location**: LocatedIn(Book, Library) - The book is located in the library.
- **Adjacency**: Adjacent(Room1, Room2) - Room1 is adjacent to Room2.

Implications: Spatial reasoning is essential for robotics, navigation systems, and geographic information systems, where understanding physical space and movement is critical.

(C) Modal Facts

Definition: Modal facts involve modalities such as necessity and possibility, allowing reasoning about different modes of truth.

Formal Representation: Modal logic, including necessity (□) and possibility (◇) operators, is used to represent modal facts.

Examples:

- **Necessity**: Necessarily(Alive(Socrates) → Mortal(Socrates)) - It is necessarily true that if Socrates is alive, he is mortal.
- **Possibility**: Possibly(Rain(Tomorrow)) - It is possible that it will rain tomorrow.

Implications: Modal reasoning is used in various domains, including epistemology (reasoning about knowledge), deontic logic (reasoning about obligations), and dynamic logic (reasoning about actions).

Summary

- **Basic Facts**: Simple, atomic statements.
- **Complex Facts**: Statements involving logical operators.
- **Terminological Facts**: Definitions and hierarchies of concepts.
- **Entailment**: Logical consequences of premises.
- **Abstract Individuals**: Conceptual, non-physical entities.
- **Other Facts**: Temporal, spatial, and modal facts.

----------X----------